altairsim — The Debugger

2026-10-16 · 277743e

A simulation of the MITS Altair 8800 and the S-100 bus

The debugger 1

- Where the processor is: `REGS` 1
- Stepping: `STEP`, `NEXT` 2
- Breakpoints: `BREAK`, `NOBREAK` 2
- Reading a block of memory: `DUMP` 5
- One byte at a time: `EXAMINE`, `DEPOSIT`, `EDIT` 6
- Disassembling: `DISASM` 8
- Symbols: `SYMBOLS`, `SHOW SYMBOLS` 9
- Searching, filling, moving: `SEARCH`, `FILL`, `MOVE`, `COMPARE` 11
- Running real bus cycles by hand: `IN`, `OUT` 12
- Asking without touching: `WHO` 12
- Looking at the bus: `SHOW BUS` 13
- The machine over time: `TRACE`, `HISTORY` 14
- What a board does: `SET … DEBUG`, `SHOW DEBUG` 16
- A copy of the session: `SET CONSOLE log` 18
- A debugging session 18
- Saving the state: `SNAPSHOT`, `RESTORE` 20
- Using an AI assistant: the MCP server 21
- About the bus 23

The debugger

This document tells you what to do in the monitor when a guest does not work correctly. It is the companion to *The Monitor*. *The Monitor* describes the `altairsim>` prompt, where you start, stop and change the machine. If the `altairsim>` prompt is new to you, read *The Monitor* first.

Both documents are companions to the *User Manual*, which describes the simulated hardware. `altairsim` simulates the MITS Altair 8800 and its S-100 bus.

A simulator must run old software. It must also show you what the machine does: which board answered, what went out on the bus, and why an interrupt did not arrive. The commands in this document show you these things. They are the main reason that `altairsim` exists. Use them when something goes wrong.

Where the processor is: `REGS`

`REGS` shows all of the processor on one line. The flags come first: carry, zero, minus, even parity and interdigit carry. Next come the register pairs, the stack pointer, the interrupt-enable flip-flop and the program counter. The last column is **the next instruction that the processor will execute**, in disassembled form.

Each time the machine stops, the monitor shows this line. For this reason, you seldom need to type `REGS`.

```
altairsim> REGS
C0Z1M0E1I0 A=00 BC=007F DE=CA01 HL=BC0E SP=BC37 IE=1 PC=CA9C  CALL CA78
```

Each register has the name that you use with `SET REG`: the pairs `BC`, `DE` and `HL`, the stack pointer `SP` and the program counter `PC`. The name that you read is the name that you type.

The line shows the registers of the processor in the machine. A Z80 has more registers than an 8080, so on a Z80 the line wraps to two lines. The Z80 adds:

- the flags `S`, `P`, `H` and `N`, beside the `C` and `Z` that both processors have
- the alternate register bank, with its own flags
- the index registers `IX` and `IY`
- the interrupt vector `I` and the interrupt mode `IM`

The **first** line has the registers that the Z80 shares with the 8080: the flags, `A`, the pairs, `SP` and `PC`. The interrupt-enable flip-flop shows as `IFF1`, which is the Z80 name for the 8080's `IE`. The line ends with the instruction at `PC`, in Z80 mnemonics.

The **second** line has the registers that only the Z80 has. These are the alternate flags, and the alternate bank `A'`, `BC'`, `DE'` and `HL'` (a prime marks each one). After these come `IX`, `IY`, the interrupt vector

`I`, the mode `IM`, and `IFF2`. `IFF2` is the copy that the Z80 keeps of `IFF1`.

```
altairsim> REGS            (on a Z80 machine)
C0Z0S0P0H0N0 A=00 BC=0000 DE=0000 HL=0000 SP=0000 IFF1=0 PC=0000  CALL PE,9A78
C0Z0S0P0H0N0 A'00 BC'0000 DE'0000 HL'0000 IX=0000 IY=0000 I=00 IM=00 IFF2=0
```

The flags are registers, and you can set them:

```
SET REG A=3F
SET REG CY=1
```

Stepping: `STEP`, `NEXT`

`STEP` runs **real bus cycles through the real instruction decoder**. It moves the machine forward by one instruction. It is the same machine that `RUN` runs, not a separate interpreter.

`STEP` shows one line for each instruction. The line shows the machine *after* the instruction ran, and the next instruction at `PC`. For example, `STEP 3` shows three lines. For a count of more than thirty-two, `STEP` shows no lines while it runs. When it stops, it tells you where the machine is.

```
STEP        one instruction
STEP 20     twenty instructions (a count, so it is decimal)
```

`NEXT` steps *over* a subroutine. At a `CALL` or an `RST`, `STEP` goes into the subroutine. It shows each instruction of the subroutine, and each instruction of the subroutines that it calls. Often you know that the subroutine works, and you want the *next* instruction in the code that you read. `NEXT` gives you that instruction.

At a `CALL` or an `RST`, `NEXT` runs the subroutine without a stop at each instruction, and stops when the subroutine returns. At any other instruction, `NEXT` does one step. `NEXT` sets a breakpoint at the return address and runs to it, as you would do by hand. The subroutine runs as it does after `RUN`, at the speed that `clock_hz` sets, and it can read the console. If it does not return, press `Ctrl-E` or `Ctrl-C` to stop it. A breakpoint *inside* the subroutine stops the machine there.

```
NEXT        step over the CALL or RST at PC, or do one step
N           the same (N is NEXT, because you type it often)
```

Breakpoints: `BREAK`, `NOBREAK`

There are three types of breakpoint:

- An **address breakpoint** (`BREAK <addr>`) watches the processor. It stops the machine when `PC` gets to the address.
- A **cycle breakpoint** (`BREAK MEM` or `BREAK IO`) watches the bus cycles, not the instructions.
- A **tape breakpoint** (`BREAK TAPE STOP`) watches a cassette deck.

Use a cycle breakpoint when you can. A memory cycle breakpoint also finds a DMA transfer, which no instruction does. It works in the same way on each processor, because it watches the backplane and not the program. If a byte changes and you do not know why, use `BREAK MEM W <addr>`. It finds the instruction or the board that writes the byte.

A cycle breakpoint stops the machine *before* the access. `PC` is on the instruction that was about to make the access. That instruction has not run: no port was read, no byte was written, and the registers have not changed. This is the same place where an address breakpoint stops. For this reason, `RUN` or `STEP` then runs the instruction from its start.

There is one exception. A cycle from a *DMA* board has no instruction to stop before. The machine stops at the end of the instruction during which the transfer occurred.

A tape breakpoint watches a device, not the program. It stops the machine when a cassette deck stops the tape by itself at the end of a load. At that time, you usually want to look at the data that the load wrote to memory. You do not need to know the end address of the loader. Set the breakpoint and run the machine. The machine stops inside the loader when the tape stops.

```
BREAK FF13            stop when PC gets to FF13
BREAK 2C00-2CFF       ...or to any address in a range
BREAK MEM W 100       stop when ANYTHING writes to 0100
BREAK IO  R 10        stop on an IN from port 10
BREAK TAPE STOP       stop when a cassette deck stops the tape after a load
BREAK                 list the breakpoints
NOBREAK 2             remove breakpoint 2 (the id is decimal, not a bus address)
NOBREAK               remove all breakpoints
```

The monitor gives breakpoints the ids 1, 2, 3 and so on, in order. When no breakpoints are left, the next id is 1 again. This is true if you removed them all with `NOBREAK`, and if you removed the last one by its id. The other ids do not change when you remove a breakpoint. For this reason, an id is not a count of the breakpoints that are set.

When a breakpoint stops the machine, the monitor tells you which breakpoint, where, and the state of the machine. An address breakpoint stops with `PC` *on* the instruction at the address. That instruction has not run yet.

```
altairsim> BREAK 2C00
breakpoint 1: pc     2C00
altairsim> RUN FF00
breakpoint 1 (pc     2C00) -- stopped at 2C00
1414 instructions, 9202 T-states.
C0Z1M0E1I1 A=C9 BC=0000 DE=2CEB HL=FFFE SP=0000 IE=0 PC=2C00  DI
```

The first line of the report names the breakpoint and the address where the machine stopped. The second line tells how many instructions and T-states ran after `RUN`. The third line is the line that `REGS` shows.

A cycle breakpoint gives the same report, with one difference. For a cycle breakpoint, *stopped at* is the address of the instruction that made the access, not the watched address. The report gives you the instruction that made the access. In this example, the breakpoint finds a write to `2C00`. `PC` is on the `STAX D` that writes the byte:

```
altairsim> BREAK MEM W 2C00
breakpoint 1: mem w  2C00
altairsim> RUN FF00
breakpoint 1 (mem w  2C00) -- stopped at FF09
4 instructions, 34 T-states.
C0Z0M0E0I0 A=F3 BC=00EB DE=2C00 HL=FF13 SP=0000 IE=0 PC=FF09  STAX D
```

Conditions: `IF`

An address breakpoint can have a condition. `BREAK <addr> IF <expr>` stops the machine only when the expression is true. The monitor tests the registers when `PC` gets to the address. Use a condition when a breakpoint occurs many times before the time that you want. Write the state that you want in the condition, and let the machine run until the condition is true.

In a condition, a word that is the name of a register means that register. For this reason, a number that starts with a letter needs a zero before it. `0A` is ten, and `A` is the accumulator. The operators are:

- `==` `!=` `<` `>` `<=` `>=` compare two values
- `&&` and `||` join two conditions
- `&` and `|` mask bits
- parentheses group

```
BREAK 100 IF A==0
BREAK 100 IF HL==8000 && Z==1
BREAK 100 IF (A&0F)==0        only when the low nibble is zero
```

A cycle breakpoint can also have a condition. `BREAK MEM W 100 IF <expr>` and `BREAK IO R 10 IF <expr>` stop only on an access where the registers make the condition true. For example, you can stop on the one write to a buffer that occurs when a counter has a given value. You can also stop on the read of a status port when a given unit is selected.

For a cycle breakpoint, `IF` tests the registers as they were when the instruction started. These are the same registers that `BREAK <addr> IF` at that instruction tests.

```
BREAK MEM W 100 IF B==0      the write to 0100 when B is zero
BREAK IO R 10 IF C==1        the IN from port 10 when unit 1 is selected
```

Tests on the byte that an `IN` read: `LOADS`

`IF` on a port read tests the registers *before* the instruction. For this reason, it cannot test the byte that the `IN` reads, because that byte is not in a register yet.

`BREAK IO R <port> LOADS <expr>` tests the condition *after* the instruction is complete. At that time, the register that the `IN` loaded holds the new value. Use `LOADS` to stop on the *value* that a port gives, and not only on the read. For example, stop when a status bit becomes 1, or when a byte is out of range.

```
BREAK IO R 10 LOADS A>7F        stop when the IN from port 10 reads a byte above 7F
BREAK IO R 08 LOADS (A&80)!=0   ...when bit 7 of the status port becomes 1
```

`IF` and `LOADS` use the same expressions. `IF` tests the start of the instruction, and `LOADS` tests the end. `BREAK IO R 10 IF A==5` tests the `A` that went *into* the instruction. `BREAK IO R 10 LOADS A==5` tests the `A` that came *out*. Only a port read loads a register, so the monitor accepts `LOADS` only on `BREAK IO R`.

Reading a block of memory: `DUMP`

Use `DUMP` to read much memory at one time. `DUMP <addr>` shows memory from the address to the **end of its page**. A `DUMP` with no address continues from there. For this reason, the rows always start at a page boundary, and the columns stay in the same place.

`DUMP` only reads memory. It runs no bus cycle, so it changes nothing.

```
DUMP 100          0100-01FF: a full page
DUMP              the next page
DUMP FF00-FF0F    the range FF00 to FF0F
DUMP 100/20       0100-011F (a length is part of the address, so it is hex)
DUMP 0 WIDTH=8    eight bytes on each line (a count, so it is decimal)
```

Each row shows the address, then the bytes in hex, then the same bytes as text. A byte that is not a printable character shows as `.`. This is the DBL boot PROM of the default machine, at `FF00`:

```
altairsim> DUMP FF00-FF3F
FF00  21 13 FF 11 00 2C 0E EB  7E 12 23 13 0D C2 08 FF  !....,..~.#.....
FF10  C3 00 2C F3 AF D3 22 2F  D3 23 3E 2C D3 22 3E 03  ..,..."/.#>,.">.
FF20  D3 10 DB FF E6 10 0F 0F  C6 10 D3 10 31 79 2D AF  ............1y-.
FF30  D3 08 DB 08 E6 08 C2 1C  2C 3E 04 D3 09 C3 38 2C  ........,>....8,
```

This memory holds code, so the text column shows mostly `.`. The text column is useful for a buffer of strings. You can read the message in the right-hand column.

One byte at a time: `EXAMINE`, `DEPOSIT`, `EDIT`

These commands are **the switches of the front panel**, and they operate as the switches do. `EXAMINE` shows one byte in hex, in ASCII and as bits. It also puts the address in the program counter, as the EXAMINE switch does. For this reason, after the byte, `EXAMINE <addr>` shows the register line and the instruction at the new `PC`. That instruction is the one that the next `STEP` runs.

An `EXAMINE` with no address shows the next byte, without the register line. This is EXAMINE NEXT on the front panel.

`DEPOSIT` runs a **real bus write**. If no board decodes the address, `DEPOSIT` tells you so. It does not tell you that it stored the byte when no board stored it.

```
EXAMINE 2C00      one byte in hex, ASCII and bits, then the register line
                  and the instruction at the new PC, ready for STEP
EXAMINE           the next byte, without the register line (EXAMINE NEXT)
DEPOSIT 100 C3 00 2C
```

`EDIT` is an interactive `DEPOSIT`. Use it to change a series of bytes, without typing the address for each byte. The prompt shows an address and the byte at that address. At the prompt, do one of these:

- Type a new value and press Enter. `EDIT` writes the value and goes to the next byte.
- Press Enter only. `EDIT` does not change the byte, and goes to the next byte.

- Type `.`. `EDIT` stops, and you are at the monitor prompt again.

`EDIT` runs the same real bus write as `DEPOSIT`. For this reason, it gives the same warning when no board decodes the address, and `EDIT <addr> ROM` writes to a PROM. `EDIT` needs you to type at it, at the prompt or through a pipe. Where nobody types (for example, in a `startup` list), use `DEPOSIT`.

```
EDIT 100          0100 C3 3E     type 3E and Enter: EDIT writes 3E and goes to 0101
                  0101 00        Enter only: the byte stays 00, and EDIT goes to 0102
                  0102 2C .      '.' stops EDIT and returns to the monitor
```

Assembling one instruction with `EDIT`

On a machine that has a processor, you can type an **instruction** where `EDIT` expects a byte. `EDIT` assembles the instruction at the address. `EDIT` does the opposite of `DISASM` (see below). For example, type `IN 10` and `EDIT` writes `DB 10`. The next prompt moves forward by the length of the instruction, not by one byte. For a two-byte instruction, the next address is two bytes on.

The numbers in an instruction use the console's number base. An `H` or `Q` after a number sets its base (hex or octal). `EDIT` does not accept labels, because it assembles one instruction and is not a full assembler. A value alone is still one byte, so you type bytes as before.

`EDIT` assembles all 8080 and 8085 instructions. For the Z80, it assembles the documented main instructions and the `CB` and `ED` instructions. It does not assemble these Z80 instructions:

- **The `IX` and `IY` indexed forms.** An indexed form has a displacement, and its `IXH` and `IXL` half-registers are undocumented.
- **The relative jumps `JR` and `DJNZ`.** A relative jump has a signed offset from the address after the jump. Its byte depends on the address that it jumps to and on the address that it jumps from.

For these instructions, `EDIT` shows *not implemented* and does not write a byte. Deposit their bytes directly, or use a `JP` to go to the same address. For a processor that `EDIT` cannot assemble for, `EDIT` accepts only bytes.

```
EDIT 100          0100 C3 IN 10        assembles DB 10, goes to 0102
                  0102 00 LXI H,FF13   assembles 21 13 FF, goes to 0105
                  0105 76 .            '.' returns to the monitor
```

Disassembling: `DISASM`

`DISASM` **peeks**. It reads memory without a bus cycle. This is important. A `read()` on a serial board *removes* a byte from its receiver. If the disassembler took the guest's input while you looked at memory, you could not trust the debugger. The commands that only *look* at memory do not change it.

```
DISASM FF00       sixteen instructions
DISASM            continue from the last one
DISASM 0-2F       the range 0000 to 002F
```

This example is the reset entry of ALTMON at `F800`, the first code that the ROM runs:

```
altairsim> DISASM F800-F811
F800  3E 03     MVI A,03
F802  D3 10     OUT 10
F804  D3 12     OUT 12
F806  3E 11     MVI A,11
F808  D3 10     OUT 10
F80A  D3 12     OUT 12
F80C  31 00 C0  LXI SP,C000
F80F  CD A5 FB  CALL FBA5
```

The code resets the 6850s of both 2SIO channels (`OUT 10` and `OUT 12`). It selects 8N2 (`MVI A,11`), sets the stack at `C000`, and calls the sign-on routine at `FBA5`. The range stops at `F811` because the bytes after it are the sign-on text. `DISASM` would decode that ASCII as instructions, because memory does not show which bytes are code.

Start `DISASM` on the first byte of an instruction. `DISASM` cannot find where an instruction starts. If you start it in the *middle* of an instruction, it decodes the operand bytes as opcodes, and the listing is wrong. For example, start the same reset code one byte late, at `F801`. The `03` that was the *operand* of `MVI A,03` becomes an instruction:

```
altairsim> DISASM F801-F812
F801  03        INX B
F802  D3 10     OUT 10
F804  D3 12     OUT 12
...
```

`INX B` is not an instruction in this ROM. A disassembler usually gets back to the real code after one or two bytes. In this example, `F802` is correct again, because `03` is a one-byte instruction. For this reason, a listing can be correct after a few lines while its first instruction is wrong. If a `DISASM` listing looks wrong,

make sure that you started at the start of an instruction. To find one, use `STEP` to get to the address, or start the range at a label that you know is correct.

`DISASM` decodes for the processor in the machine. The same bytes are different instructions on different processors. `DISASM` decodes them for the processor in the machine, and shows them in the assembly language of that processor. On a Z80, some bytes that are undefined on an 8080 are real instructions. `DISASM` shows them in Zilog mnemonics (`LD`, not the 8080's `MVI` and `MOV`):

```
altairsim> DISASM 100-10A           (on a Z80 machine)
0100  ED B0     LDIR
0102  CB 27     SLA A
0104  18 FE     JR 0104
0106  10 FC     DJNZ 0104
0108  DD 7E 05  LD A,(IX+05)
```

The 8080 has no `ED`, `CB` or `DD` prefix. On an 8080, `DISASM` marks each undefined byte (`??=`), and decodes the other bytes as 8080 instructions that have no relation to the Z80 code:

```
altairsim> DISASM 100-10A           (the same bytes, on an 8080 machine)
0100  ED        ??= ED  *CALL
0101  B0        ORA B
0102  CB        ??= CB  *JMP
0103  27        DAA
0104  18        ??= 18  *NOP
0105  FE 10     CPI 10
0107  FC DD 7E  CM 7EDD
010A  05        DCR B
```

Symbols: `SYMBOLS`, `SHOW SYMBOLS`

The commands above use hex addresses. If you load the symbols from an assembler, you can use names. For example, type `BREAK START` and not `BREAK 0100`, or `DUMP MSG/20`, or `EXAMINE BDOS`. You can type a symbol in each place where you can type an address, and in a `BREAK … IF` condition.

```
SYMBOLS LOAD prog.SYM             a symbol table
SYMBOLS LOAD ALTMON.PRN           ...or an assembler listing
BREAK START
DUMP MSG/20
BREAK 200 IF HL==STACK
SHOW SYMBOLS                      all of the symbols
SHOW SYMBOLS SIO*                 the symbols that match a pattern
SYMBOLS CLEAR                     remove all symbols
```

This is the same disassembly as before, with `ALTMON.PRN` loaded. The range starts at a symbol, and the listing shows names where it can. `DISASM` is the only command that shows symbols in its *output*. `DUMP` still shows hex and ASCII, because a block of data does not show which bytes are addresses.

```
altairsim> SYMBOLS LOAD ALTMON.PRN
96 symbol(s) from ALTMON.PRN
altairsim> DISASM MONIT-F811
MONIT:
F800  3E 03     MVI A,03
F802  D3 10     OUT 10
F804  D3 12     OUT 12
F806  3E 11     MVI A,11
F808  D3 10     OUT 10
F80A  D3 12     OUT 12
F80C  31 00 C0  LXI SP,SPTR
F80F  CD A5 FB  CALL DSPMSG
```

`MONIT` is `F800`, so the range starts at `F800`. A name works as an address in each command. The listing also changes in two ways:

- **A program label** is on its own line above its address, as in an assembler listing. `MONIT:` is above `F800`. For this reason, you can see where a jump goes to.
- **A 16-bit operand** shows as a name. `CALL FBA5` becomes `CALL DSPMSG`, and `LXI SP,C000` becomes `LXI SP,SPTR`.

These two changes use different symbols, for a reason:

- **The label line uses program labels only.** `SPTR` is an **`EQU`** (the listing marks it with `=`), so it never shows as a label line. A constant that has the same value as a code address is not a code address. The same rule stops `0005` from showing a false `BDOS:` label.
- **An operand can use an `EQU`.** An operand is a value that the instruction *points at*. An `EQU` that holds an address is the name that you want to read there. For this reason, `LXI SP,SPTR` shows its target, although `SPTR` is an `EQU`. For the same reason, `CALL 0005` shows as `CALL BDOS`.
- **A program label is used before an `EQU`** when the two have the same value.

Only a 16-bit operand shows as an address. A byte value stays a number. `MVI A,03` does not become a symbol, even if an `EQU` has the value three, because a two-digit value is a count and not an address.

> **Try it.** `examples/debugger/` is a 46-byte program for these rules. It has a `.PRN` for `SYMBOLS LOAD`, a `.HEX` for `LOAD`, and a `README`. The `README` starts with a `DISASM` that shows symbols. Next, it steps through the program, stops at a label, and runs the program until it prints. You can see each rule above in the example, including the `EQU` operand and the byte value that stays a number.

The two types of symbol file

A `.SYM` file is a list of names and values. Two toolchains write one:

- The Digital Research `MAC` and `RMAC` assemblers write all the symbols, for `SID`.
- The Microsoft **L80** linker writes one with the correct switches. `L80`'s `/M` shows a *map* on the console, but `filename/N/Y/E` writes a real `filename.SYM`. An `L80` `.SYM` has **only the global names** (the `PUBLIC` names). The local labels and `EQU`s of a module are not in it. For those, use the assembler's listing.

A `.PRN` or `.LST` file is the listing from an assembler: CP/M `ASM`, Microsoft `M80`, or `MAC`. A listing gives more than a `.SYM` file, because it marks each `EQU`. For this reason, the debugger can tell a constant from a program label. Only program labels show as label lines, so `0005` does not show as `BDOS:`.

The addresses must be absolute. A relocatable `M80` listing marks its addresses, and `SYMBOLS LOAD` refuses it and names the line. Link the program and load the `.SYM`, or assemble it to an absolute origin. The linker writes a `.SYM` after it links, so the addresses in a `.SYM` are always absolute.

Symbols belong to you, not to the machine

Like a breakpoint, the symbol table is part of the debugger, not part of a board. It stays after `RESET`, `POWER` and `CONFIG LOAD`. `SYMBOLS CLEAR` removes the symbols, as `NOBREAK` removes the breakpoints.

When you load a second file, `SYMBOLS LOAD` **merges** the two. If a name is in both files, the newer value is used, and the command tells you how many names changed. `SYMBOLS LOAD <file> REPLACE` removes the old symbols first. A machine file can load a symbol file in its `startup`. `CONFIG SAVE` saves the file name back to the machine file, not the symbols. It does the same for a built-in ROM.

A name is used before a hex number. If a symbol looks like a number, such as `FACE` or `BEEF`, the monitor uses the symbol. To type the number, write `0FACE` or `$FACE`. The same zero tells the register `A` from the number `0A`.

Searching, filling, moving: `SEARCH`, `FILL`, `MOVE`, `COMPARE`

These commands operate on blocks of memory. `COMPARE` can use a file as its second operand. Use it to compare the data that the machine loaded with the data that you wanted to load.

```
SEARCH 0-FFFF C3 00 2C      find these bytes
SEARCH 0-FFFF "BDOS"        ...or this string
FILL 100-1FF 00
MOVE 100-1FF 2000
COMPARE 100-1FF 2000        ...or compare with a file
```

Running real bus cycles by hand: `IN`, `OUT`

`IN` and `OUT` run real bus cycles. **`IN` runs an input cycle on the bus, with each effect that a real cycle has.** For example, it removes a character from the receiver of a UART, and it moves the sector counter of a disk controller forward. Use `IN` and `OUT` to operate a board as the guest does, without writing a guest program.

```
IN  10            run a real IN cycle on port 10
OUT FF 55         run a real OUT cycle
```

Asking without touching: `WHO`

`WHO` tells you which **board** *would* answer an address or a port. It gives the board's id. **`WHO` runs no cycle, and it changes nothing.** Use it when `IN 10` gives `FF`, and you do not know if `FF` is data or if no board answered.

The console 2SIO decodes port 10, so `WHO` names it. A read and a write can go to different boards, so `WHO` shows each one:

```
altairsim> WHO IO 10
port 10 IN:  sio0
port 10 OUT: sio0
```

For a port that no board decodes, `WHO` shows `nobody`. This tells you that the `FF` was a floating bus, not data:

```
altairsim> WHO IO 20
port 20 IN:  nobody (an IN here reads FF)
port 20 OUT: nobody (an OUT here goes nowhere)
```

`WHO` also shows contention and `PHANTOM*`. Use it to find two boards that decode the same address, or a board that disables another board. On the default machine, the DBL boot ROM is part of `mem0` at `FF00`. It asserts `PHANTOM*` to disable the RAM below it. For this reason, a read at `FF00` comes from the ROM, and no board takes a write. `WHO` shows both:

```
altairsim> WHO FF00
FF00 read  mem0   [PHANTOM* asserted]
FF00 write nobody -- floats to FF (a write here is simply gone)  [PHANTOM* asserted]
```

`WHO <addr>` asks about a memory address. `WHO IO <port>` asks about a port.

Looking at the bus: `SHOW BUS`

`WHO` asks about one address. `SHOW BUS` shows all of the backplane at one time.

`SHOW BUS IRQ` is the only command that shows the interrupt wiring. You cannot see this wiring in any other way. If a board is set to an interrupt line that nothing reads, nothing shows the fault. The guest does not get its interrupt, and there is nothing to examine. `SHOW BUS IRQ` shows you this fault.

Use `SHOW BUS CONTENTION` when a machine that you built does not operate correctly and you do not know why. Two boards that decode the same port are a real hardware fault. The simulator does not choose one of the two boards for you.

`SHOW BUS` with no view prints three sections: `MEMORY`, `I/O` and `INTERRUPTS`. The `INTERRUPTS` section is the short form of `SHOW BUS IRQ`. Run it first on a machine that you built:

```
altairsim> SHOW BUS
MEMORY
  0000-0FFF  mem0     ram
  unmapped: 1000-FFFF  (floats to FF)
I/O
  FF         fp0      IN  --     SENSE switches SA8..SA15 -> D0..D7
  00         sio0     IN  OUT    COM2502 -- status / interrupt enables
  01         sio0     IN  OUT    COM2502 -- data
  06         acr0     IN  OUT    88-SIO B UART -- status / interrupt enables
  07         acr0     IN  OUT    88-SIO B UART -- data, via the modem, to the cassette
INTERRUPTS
  CPU     INTE off         interrupts are DISABLED; nothing will be acknowledged
  pINT    idle             pin 73
  88-VI   (none)           nothing watches VI0-VI7; an acknowledged interrupt
                           floats to FF, which the 8080 executes as RST 7
```

To see one section, or more detail, add a view:

```
SHOW BUS              memory, ports and interrupts, all at one time
SHOW BUS MAP          which board decodes each memory address, and which addresses float
SHOW BUS IO           which board decodes each port
SHOW BUS IRQ          the eight interrupt lines: which board is set to each, and which assert
SHOW BUS CONTENTION   the addresses that two boards answer
```

`SHOW BUS MAP` lists only the memory that answers now. A boot PROM that the guest has switched out is not in the list, and `WHO` gives the same answer for an address in it. `SHOW MACHINE` shows how each board is built, and it lists the PROM always.

The machine over time: `TRACE`, `HISTORY`

`WHO` and `SHOW BUS` show the backplane as it is *now*. `REGS` and `STEP` show the machine as it is *now*. `TRACE` and `HISTORY` show the machine over *time*. Use them when the fault is not where the machine stopped, but in the instructions before the stop.

`HISTORY` records the machine all the time. While the machine runs, it fills a fixed-size buffer with the most recent instructions. When a breakpoint stops the machine, or when the guest runs code that it must not run, the instructions before the stop are already recorded. You do not need to turn `HISTORY` on.

`HISTORY` shows the last sixteen **instructions**, oldest first. `HISTORY <n>` shows the last *n*. This is the **instruction history**. Each line is the same as a `STEP` line: the registers and flags as they were, and the instruction that was next. `HISTORY` decodes each instruction from the bytes that *ran* at that address. For this reason, it shows code that changed itself correctly.

The **bus history** records the bus cycles, for when the instruction history is not enough. `HISTORY BUS` shows it. It has no registers and no mnemonics. It has the columns `T-STATE`, `TYPE`, `ADDR` and `DATA`, and then **the board that drove the cycle and the board that answered it**:

- **The driver.** The processor drives most cycles, so the column shows `cpu`. For a **DMA transfer, the column names the board** that took the bus. A DMA transfer is not an instruction, so it shows in the bus history and not in the instruction history.
- **The board that answered.** This is the board that decoded the address. The column shows `--` when no board decoded it and the read floated to `FF`.

For example, a guest that reads a port that no board decodes shows `cpu -> --`. A DMA controller that fills a frame buffer shows `dazzler -> mem0`. `HISTORY CPU` is the same as `HISTORY`.

```
HISTORY              the last 16 instructions
HISTORY 100          the last hundred (a count, so decimal)
HISTORY BUS          the last 16 bus cycles
HISTORY BUS 100      the last hundred cycles
```

`TRACE` records each cycle when it occurs, on the console or in a file. `TRACE` does not watch the processor. It watches the same cycles that each board sees, as a cycle breakpoint does. For this reason, it works in the same way on each processor.

A `MASK` limits the trace to the types of cycle that you name. With no mask, `TRACE` records all cycles. With a mask, it records a cycle that is one of the types in the list. There are five types:

- **`IN`**: an I/O read, an `IN` from a port.
- **`OUT`**: an I/O write, an `OUT` to a port.

- **`IRQ`**: an interrupt-acknowledge cycle. This is the `INTA` cycle in which the processor reads the instruction of the device that interrupts.
- **`DMA`**: each cycle that a bus master drove, *of each type*. `MASK=DMA` gives all of a DMA transfer, not only its reads or its writes.
- **`CONTENTION`**: a cycle that two or more boards answered, because they decode the same address. `SHOW BUS CONTENTION` reports the same fault. `TRACE` records it when it occurs.

```
TRACE ON                    each cycle, on the console
TRACE ON run.log            ...in a file
TRACE ON MASK=IN,OUT        only the port cycles
TRACE ON MASK=IRQ,DMA       only interrupts and DMA
TRACE OFF                   stop the trace
```

Tracepoints: tracing one part of a program

A trace of a full program is very long. A mask limits the trace to *types* of cycle. Often you want a *place* in the program instead, such as one subroutine.

For this, use a **tracepoint**. Add `TRACE ON` or `TRACE OFF` to a `BREAK`, and it becomes a tracepoint. A tracepoint does not stop the machine. It starts or stops the trace, and the machine continues. Use two tracepoints to trace a region:

```
altairsim> BREAK 2C00 TRACE ON     start the trace when PC gets to 2C00
altairsim> BREAK 2C40 TRACE OFF    ...and stop it at 2C40
altairsim> RUN FF00
```

A `TRACE ON` tracepoint traces the instruction *at* its address. A `TRACE OFF` tracepoint does not. The region is `[on, off)`. This is the half-open range that contains the instructions of the subroutine.

A tracepoint can also be a cycle breakpoint, as `IF` can. The cycle that starts the trace is the *first line* of the trace, not the line before it. For this reason, the trace shows why it started.

```
altairsim> BREAK MEM W 2000 TRACE ON    start the trace at the cycle that writes 2000
```

A tracepoint can have an `IF` condition, and it still does not stop the machine:

```
altairsim> BREAK 200 IF HL==8000 TRACE ON
```

`TRACE` sets where the trace goes, not the tracepoint. A tracepoint traces to the console, until you give `TRACE` a file. To send the trace to a file, set up `TRACE` first. `TRACE OFF` stops the trace, but *keeps the file and the mask*. A tracepoint then starts the trace again with that file and mask.

```
altairsim> TRACE ON run.log MASK=DMA    set the file and the mask (this starts the trace)
altairsim> TRACE OFF                    stop, but keep the file and the mask
altairsim> BREAK 2C00 TRACE ON          set the region
altairsim> BREAK 2C40 TRACE OFF
altairsim> RUN FF00                     run.log gets only the DMA cycles from 2C00 to 2C40
```

The `BREAK` list shows the tracepoints with the breakpoints. Their `hits` column counts how many times each one occurred. A tracepoint and a breakpoint can be at the same address. When the machine gets to the address, the trace starts or stops *and* the machine stops.

What a board does: `SET … DEBUG`, `SHOW DEBUG`

`TRACE` and `HISTORY` watch the *bus*: the cycles, addresses and data that all boards share. Some parts of the machine can also report what they do, in their *own* terms. For example, a floppy controller moves its heads and reads a sector, a serial chip receives a byte, and a socket answers a call. These reports come from **diagnostic channels**. Each part that has a diagnostic channel has a name for it and a small set of **debug flags**. You turn on the flags for the reports that you want.

`SHOW DEBUG` lists each diagnostic channel, its debug flags, and where the reports go:

```
altairsim> SHOW DEBUG
debug  (runtime diagnostics -- the sink and flags do not survive CONFIG SAVE)

  sink  stderr   -- SET CONSOLE DEBUG=stderr|stdout|<file>

  CHANNEL  FLAGS  (an enabled flag is UPPER-CASE)
  -------  -------------------------------------
  socket   connect
  6850     serial
  dsk0     SECTOR seek

  SET <channel> DEBUG=<flag>[,<flag>]  enables;  NODEBUG=<flag> disables;
  DEBUG=all / DEBUG=none turn every flag on / off.
```

A debug flag in capital letters is on. `DEBUG=` turns flags on, and `NODEBUG=` turns them off. Each one changes only the flags that you name. Each one accepts a list with commas, and the words `all` and `none`:

```
SET dsk0 DEBUG=sector,seek     report the sector reads and the head moves
SET dsk0 NODEBUG=seek          stop the head-move reports, keep the sector reads
SET dsk0 DEBUG=all             all the reports of this board
SET dsk0 NODEBUG=all           no reports from this board
```

The name before `DEBUG` is the diagnostic channel. Usually it is the id of a board, such as `dsk0` above. A shared chip and the socket layer also have diagnostic channels (`6850` and `socket`). For this reason, the same command reaches parts of the machine that are not boards. If one flag in the list is not known, the monitor refuses the command and changes *nothing*. For this reason, a typing error in a list does not leave half of the list set.

Each report line starts with **the address of the instruction that operated the board**, and then the name of the diagnostic channel. For this reason, a report line shows you the code that operates the board:

```
2C38  dsk0: sector drive=0 track=0 sector=1
007F  dsk0: seek drive=0 track=0 -> 1
```

At the monitor prompt, the machine is stopped and no instruction operates the board. The address column then shows `----`.

All the diagnostic channels send their reports to one place. Set it with `SET CONSOLE DEBUG=`:

```
SET CONSOLE DEBUG=stderr       the default
SET CONSOLE DEBUG=stdout
SET CONSOLE DEBUG=trace.log    add to the end of a file
```

altairsim writes each line to the file when the line is made. To see the lines while the machine runs, type `tail -f trace.log` in a second window.

`Tab` completes each part of these commands:

- the channel name after `SET`
- `DEBUG` or `NODEBUG` after the channel
- the debug flags, and `all` and `none`, after the `=`

`CONFIG SAVE` does not save these settings. You turn a diagnostic channel on to watch a problem, and it is not part of the machine. For this reason, a machine file that you save while you debug does not turn the reports on for each run after it.

A copy of the session: `SET CONSOLE log`

`TRACE` records the bus, and a diagnostic channel records what a board does. `SET CONSOLE log` records the *console*: all that you saw. It writes the guest's output and the keys that you typed to a host file, when they occur. You can read the session again later, or give it to a person who was not there.

```
SET CONSOLE log=session.txt     start to copy the session to a file
SET CONSOLE log=off             stop (an empty path does the same)
```

`log` adds to the end of the file. If you set `log` to the same file two times in a session, the second copy goes after the first, and the first stays. The log is a diagnostic tool, like `TRACE` and the diagnostic channels, and it is not part of the machine. For this reason, `CONFIG SAVE` does not save it. A machine file that you save while you log a session does not turn the log on for each run after it.

A debugging session

This short session uses the commands above on the `altmon` machine. ALTMON prints its `ALTMON 1.3` banner at reset. It prints the banner without a pointer to the text, and the session finds out how. Set a breakpoint at the sign-on routine `FBA5`, and run from the reset entry:

```
altairsim> BREAK FBA5
breakpoint 1: pc     FBA5
altairsim> RUN F800

breakpoint 1 (pc     FBA5) -- stopped at FBA5
8 instructions, 81 T-states.
C0Z0M0E0I1 A=11 BC=0000 DE=0000 HL=F81F SP=BFFE IE=0 PC=FBA5  POP H
```

The breakpoint shows the registers, so you do not need `REGS`. `PC` is on `POP H`, the first instruction of the routine. Disassemble the routine:

```
altairsim> DISASM
FBA5  E1        POP H
FBA6  7E        MOV A,M
FBA7  CD 48 FB  CALL FB48
FBAA  B6        ORA M
FBAB  23        INX H
FBAC  F2 A6 FB  JP FBA6
FBAF  CD 46 FB  CALL FB46
FBB2  E9        PCHL
FBB3  CD BC FB  CALL FBBC
FBB6  FE 1B     CPI 1B
FBB8  C8        RZ
FBB9  C3 48 FB  JMP FB48
FBBC  DB 10     IN 10
FBBE  0F        RRC
FBBF  D2 BC FB  JNC FBBC
FBC2  DB 11     IN 11
```

This is how the routine works. `POP H` puts the routine's *own return address* in `HL`. The text is in memory directly after the `CALL FBA5`, so `HL` now points at the text. The loop is:

1. `MOV A,M` gets a character.
2. `CALL FB48` prints it.
3. `ORA M` tests the byte.
4. `INX H` moves to the next byte.
5. `JP FBA6` goes back to the start of the loop.

Step into the routine, and see the banner come out one character at a time:

```
altairsim> STEP 20
C0Z0M0E0I1 A=11 BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBA6  MOV A,M
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBA7  CALL FB48
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB48  PUSH PSW
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB49  IN 10
C0Z0M0E0I1 A=02 BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB4B  ANI 02
C0Z0M0E0I0 A=02 BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB4D  JZ FB49
C0Z0M0E0I0 A=02 BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB50  POP PSW
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB51  ANI 7F
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB53  OUT 11
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB55  RET
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBAA  ORA M
C0Z0M0E0I0 A=0D BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBAB  INX H
C0Z0M0E0I0 A=0D BC=0000 DE=0000 HL=F813 SP=C000 IE=0 PC=FBAC  JP FBA6
C0Z0M0E0I0 A=0D BC=0000 DE=0000 HL=F813 SP=C000 IE=0 PC=FBA6  MOV A,M
C0Z0M0E0I0 A=0A BC=0000 DE=0000 HL=F813 SP=C000 IE=0 PC=FBA7  CALL FB48
C0Z0M0E0I0 A=0A BC=0000 DE=0000 HL=F813 SP=BFFE IE=0 PC=FB48  PUSH PSW
C0Z0M0E0I0 A=0A BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB49  IN 10
C0Z0M0E0I0 A=00 BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB4B  ANI 02
C0Z1M0E1I0 A=00 BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB4D  JZ FB49
C0Z1M0E1I0 A=00 BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB49  IN 10
```

`POP H` set `HL` to `F812`, the byte directly after the `CALL`. The first character is `0D`, a carriage return. `FB48` is the console output routine. It reads the 2SIO status (`IN 10` and `ANI 02`) until the transmitter is ready. Next, `OUT 11` sends the byte, and the routine returns.

In the loop, `INX H` moves `HL` to `F813`. The next character, `0A` (line feed), goes through the same steps. The loop continues until a byte with its high bit set marks the end of the text. After twenty steps, the first two characters of the banner are on the console.

Saving the state: `SNAPSHOT`, `RESTORE`

`SNAPSHOT` writes all the state of the machine to a file. This includes the processor, the clock, and the registers, RAM and latches of each board. `RESTORE` reads the file back into a machine that has the same boards. Use them to save the machine at one moment and go back to that moment later.

To save the state, change memory, and get the state back:

1. Type `SNAPSHOT before.snap` to save the state.
2. Type `DEPOSIT 100 00 00 00 00` to change four bytes.
3. Type `RESTORE before.snap` to get the state back.
4. Type `DUMP 100-103`. The four bytes have their old values again.

```
altairsim> SNAPSHOT before.snap
snapshot written to before.snap
altairsim> DEPOSIT 100 00 00 00 00      change four bytes
altairsim> RESTORE before.snap
restored from before.snap
altairsim> DUMP 100-103                 the old bytes are back
0100  DE AD BE EF                                  ....
```

A snapshot holds the *state* of the machine, not its *configuration*. The file does not hold the boards. `RESTORE` loads the state into the machine that you have now. If the file does not match the machine (the same boards, ids and order), `RESTORE` refuses it and does not change the machine. Make the machine first, with the machine file or with `CONFIG LOAD`. After that, restore the snapshot into it.

Using an AI assistant: the MCP server

Each command in this document is a command that you type. An **AI assistant** can also use it. Start the machine with `--mcp`, and not at a terminal:

```
$ altairsim <machine> --mcp
```

The assistant then gets the debugger as a set of tools. It can set a breakpoint, run to it, read the registers, disassemble, step, dump memory and read the bus history. It uses the *same* machine that the monitor operates. For this reason, you do not need to learn the commands. You can tell the assistant the problem in one sentence, and the assistant operates the machine for you.

The *User Manual* has a full chapter, **The MCP server**. That chapter also tells how to connect the server to other clients.

Setting up the assistant

The example uses Claude Code. You do these steps one time.

1. Go to the directory that holds your machine file. The relative paths and the host bridge then find their files there.

2. Register the server, so that the assistant can get to the machine:

   ```
   $ claude mcp add altairsim -- altairsim <machine> --mcp
   ```

3. Make sure that the server is registered and that the assistant can get to it:

```
$ claude mcp list
```

4. Copy `DRIVING-WITH-AI.md` and the `skills` folder from the package into the same directory. These files are for the assistant, not for you. `DRIVING-WITH-AI.md` tells the assistant how to use the tools, and how to boot and debug with them. The files in `skills` have the rules for programs that run in the machine, such as the assembler.

```
$ cp /path/to/DRIVING-WITH-AI.md .
$ cp -R /path/to/skills .
```

5. Start `claude` in the directory, and tell the assistant to read the file before you give it a task:

```
$ claude
> Read DRIVING-WITH-AI.md, then use the altairsim MCP tools for what follows.
```

After this, you talk to the assistant, not to the server.

What to tell the assistant

Give the assistant the full task in plain words. Name the tools, so that the assistant uses the simulator and does not guess:

> *Use the altairsim MCP tools. Boot the machine and run `HELLO.COM`. It must print `HELLO, WORLD`, but it prints `ELLO, WORLD`. Find the bug and fix the source.*

> *My loader stops before it gets to the prompt. Boot with the MCP tools, and set a breakpoint at `2C00`, where the loader moves itself. Single-step from there, and tell me where it goes wrong.*

> *Something writes over the BIOS at `E400`. Set a memory-write breakpoint there, and run until it stops the machine. Show me the instruction and the registers that wrote the byte.*

What the assistant does

The assistant does the same steps that you would do, one tool call at a time. For the first task above:

1. It boots the machine, runs the program, and sees `ELLO, WORLD`.
2. It sets a breakpoint at `0100`, runs to it, and disassembles the code.
3. It finds an `INX H` that moves the pointer *before* the program reads the first character. For this reason, the program does not print the `H`.
4. It corrects the source and assembles it again on the CP/M disk.
5. It runs the program again, and sees `HELLO, WORLD`.

You see the reasoning and the fix, and you type none of the commands.

The assistant can also use each monitor command. It can run a monitor command and read the reply. For example, a conditional breakpoint (`BREAK 200 IF HL==8000`) or an octal dump is one tool call, as it is one command for you.

`examples/ai-mcp/` is the first task above, ready to use. It has a CP/M machine, and a `HELLO.ASM` that has one bug, put there for the example. It also shows the session that the assistant runs to find and fix the bug. Use this example to see all the steps work.

About the bus

These are not commands. They are facts about the backplane that explain a reading that looks wrong.

When `FF` is not data

`FF` is a good byte. It is `RST 38` (`RST 7`), and it is the value −1. Usually, when a read gives `FF`, a board put `FF` there. `FF` is also what the bus gives when *no board answers*. Learn to tell these two cases apart.

An `IN` from a port that no board decodes gives `FF`. A read from an address that no board answers also gives `FF`. This is not an error code, and `altairsim` did not choose it. It is what a **floating bus** gives. No board drives the data lines, so they stay high, and the processor reads eight ones. A real Altair does the same.

This has an important result. Suppose that a machine has no interrupt-vector board, and a board asserts the interrupt line. During the acknowledge cycle, no board drives the data bus, so the processor reads `FF`. `FF` is `RST 7`. Nobody programmed this. It is what the hardware does, and it is the reason that the interrupt vector on an Altair with no vector board is `RST 7`.

When a read gives an `FF` that you did not expect, use `WHO`. It tells you if a board answered with that byte, or if no board answered and the bus floated.